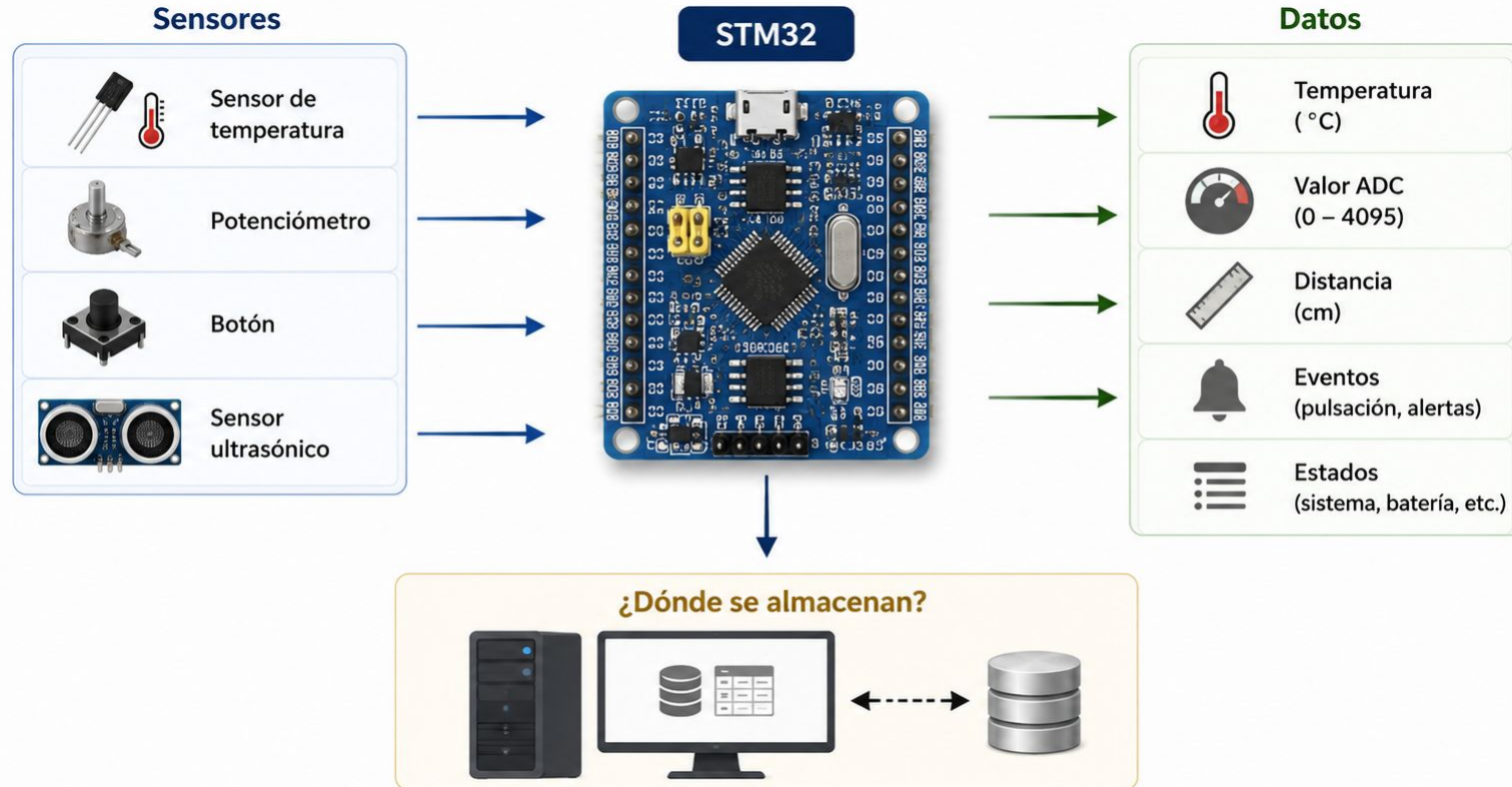


Bases de datos

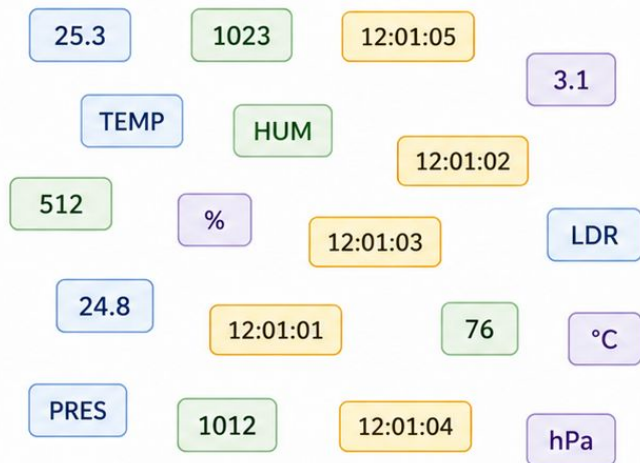
Clase 13 - 09/06/2026

¿Por qué hablar de bases de datos en SE?



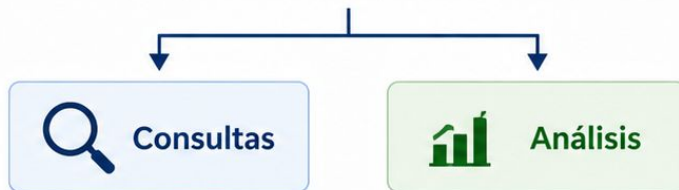
¿Qué es una base de datos?

Datos sueltos



Datos organizados

tiempo	sensor	valor	unidad
12:01:01	TEMP	24.8	°C
12:01:02	HUM	76	%
12:01:03	LDR	512	-
12:01:04	PRES	1012	hPa
12:01:05	TEMP	25.3	°C
12:01:05	ADC	1023	-
12:01:06	HUM	74	%
...	...	...	...



Tres formas de almacenar datos en SE

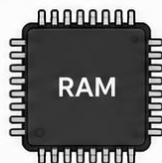
1 Variable en RAM

Valor temporal dentro del microcontrolador



Solo existe mientras el sistema está encendido

Microcontrolador (RAM)



```
uint32_t adc_value  
= 2048;
```

Valor actual de una lectura ADC

Ejemplo

```
uint32_t adc_value = 2048;
```

Se usa para cálculos, decisiones, control, etc. en tiempo real.

2 Archivo CSV

Archivo de texto con datos en formato de filas



Fácil de generar y abrir en cualquier aplicación



Memoria
(SD, Flash, etc.)



datos.csv

tiempo	sensor	valor	unidad
12:01:01	TEMP	24.8	°C
12:01:02	HUM	76	%
12:01:03	LDR	512	-
12:01:04	PRES	1012	hPa
...			

Ejemplo

datos.csv

Formato simple y ligero.
Ideal para registrar datos y analizarlos luego en una PC.

3 Base de datos

Datos estructurados en tablas con consultas y relaciones



Permite consultas, filtros, análisis y relaciones



Microcontrolador



Comunicación
(USB, Wi-Fi, BLE, etc.)



Base de datos
en PC / Servidor

Tabla: mediciones

id	tiempo	sensor	valor	unidad
1	12:01:01	TEMP	24.8	°C
2	12:01:02	HUM	76	%
3	12:01:03	LDR	512	-
4	12:01:04	PRES	1012	hPa
...				

Ejemplo

Consulta:

```
SELECT tiempo, valor  
FROM mediciones  
WHERE sensor = 'TEMP'  
ORDER BY tiempo;
```

Permite buscar, filtrar y analizar grandes volúmenes de datos.



Idea clave:

Desde un valor temporal en RAM, hasta archivos CSV y bases de datos estructuradas, la elección depende de la necesidad de persistencia, análisis y escala.



RAM: rápido y temporal

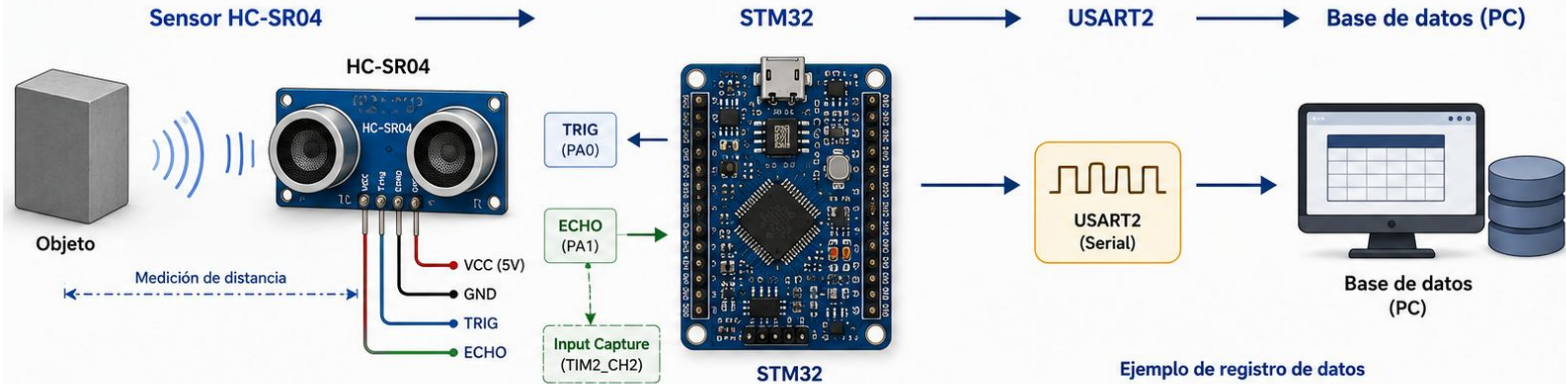


CSV: simple y portátil



BD: estructurado y consultable

Registro de datos del sensor HC-SRo4



 ¿Cómo funciona?

1. STM32 envía un pulso de 10 μ s por TRIG.
2. HC-SR04 emite 8 ciclos ultrasónicos y levanta ECHO.
3. STM32 mide el ancho del pulso en ECHO usando Input Capture (TIM2_CH2).
4. Se calcula la distancia y se envía por USART2.

 Cálculo de distancia

$$\text{distancia_cm} = \frac{\text{echo_us} \times 0.0343}{2}$$

(sound speed ≈ 343 m/s)

echo_us: tiempo de ida y vuelta del eco



Importante:

El uso de Input Capture permite medir con alta precisión el tiempo del eco, lo que se traduce en mediciones de distancia más precisas y confiables.

Ejemplo de registro de datos

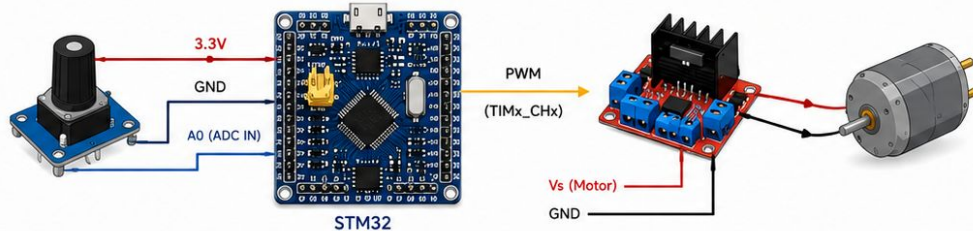
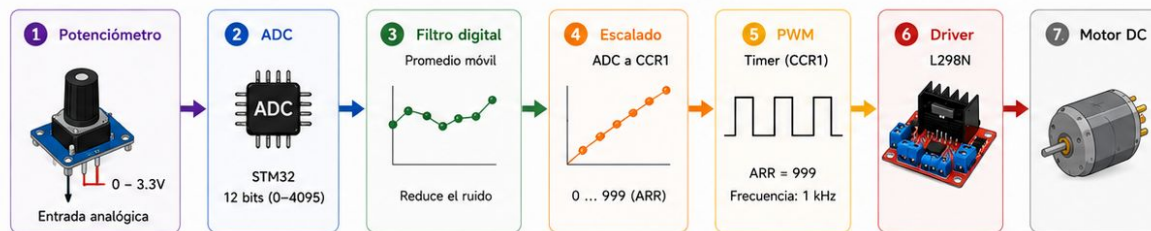
tiempo_ms	echo_us	distancia_cm	estado
0	5824	99.8	OK
200	3440	58.9	OK
400	2480	42.6	OK
600	1712	29.4	OK
800	1208	20.7	OK
1000	680	11.7	OK
1200	0	---	SIN ECO



Los datos se registran con marca de tiempo para su análisis y visualización.

SE con potenciómetro, ADC, PWM y motor DC

Potenci6metro → ADC → Filtro promedio m6vil → Escalado a CCR1 → PWM → Driver L298N → Motor DC



Datos de ejemplo I (ARR = 999)

tiempo_ms	adc_filtrado (0-4095)	tension_v (0-3.3)	ccr1 (0-999)	duty_percent (%)	direccion
0	0	0.00	0	0.0	STOP
100	128	0.10	31	3.1	CW
200	512	0.41	125	12.5	CW
300	1024	0.82	250	25.0	CW
400	1536	1.24	375	37.5	CW
500	2048	1.65	500	50.1	CW
600	2560	2.06	625	62.6	CW
700	3072	2.47	750	75.1	CW
800	3584	2.88	875	87.6	CW
900	3968	3.20	968	96.8	CW
1000	4095	3.30	999	100.0	CW

Valor filtrado → escalado → El duty depende de CCR1 (duty = CCR1 / 999 × 100)



Filtro: Promedio m6vil

- N = 8 muestras
- Promedio:

$$y[k] = \frac{1}{N} \sum_{i=0}^{N-1} x[k-i]$$
- x[k]: muestra ADC actual
- y[k]: valor filtrado



Escalado ADC a CCR1

Convierte el rango ADC (0-4095) al rango del timer (0-999)

$$CCR1 = \frac{adc_filtrado \times 999}{4095}$$

(Resultado entero entre 0 y 999)



PWM y duty cycle

- ARR = 999 → Perodo = ARR + 1 = 1000 cuentas
- Frecuencia PWM = 1 kHz (ejemplo)

$$duty_percent = \frac{CCR1}{999} \times 100$$



Resumen del flujo

- La se1al del potenci6metro se convierte en velocidad del motor mediante:
- Digitalizaci6n (ADC)
 - Filtro (promedio m6vil)
 - Escalado al timer (CCR1)
 - Generaci6n de PWM
 - Control del motor con L298N

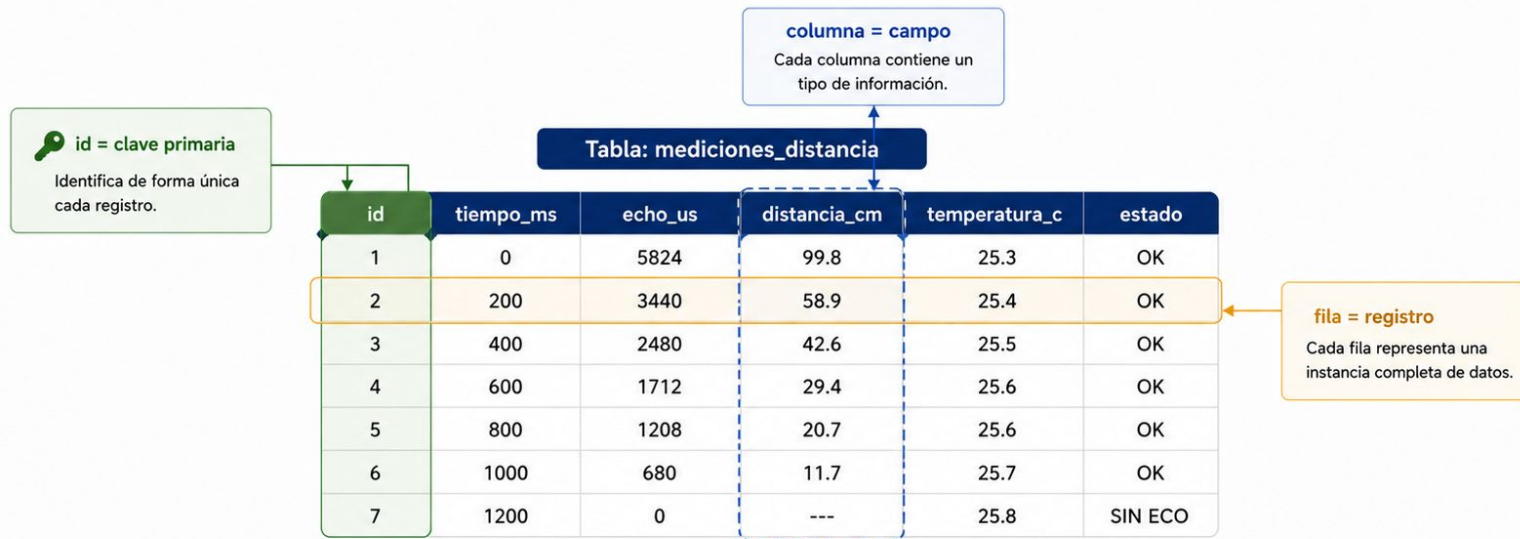


Rangos clave:

ADC: 0-4095 (12 bits) | CCR1: 0-999 (ARR = 999) | Duty: 0-100 % | PWM: 1 kHz (ejemplo)

BD relacional

Una base de datos relacional organiza la información en **tablas** formadas por filas y columnas.



Tipos de dato comunes

INT

Números enteros

123

FLOAT / REAL

Números decimales

3.14

VARCHAR

Texto de longitud variable

'OK'

DATETIME

Fecha y hora

📅 ⌚

— Clave primaria

- - - Columna (campo)

— Fila (registro)



Tipo de dato

Define el tipo de información que puede almacenar cada campo.

Introducción a SQL



SQL (Structured Query Language) permite definir estructuras de datos y almacenar mediciones generadas por un sistema embebido.

Ejemplo de instrucción SQL

```
1 CREATE TABLE mediciones_distancia (  
2   id INTEGER PRIMARY KEY,  
3   tiempo_ms INTEGER,  
4   echo_us INTEGER,  
5   distancia_cm REAL,  
6   valido INTEGER  
7 );  
8
```



¿Qué hace esta instrucción?

Crea una tabla llamada mediciones_distancia para almacenar las mediciones de un sensor ultrasónico.



Notas

- id es la clave primaria: identifica de forma única cada registro.
- tiempo_ms: tiempo desde el inicio de la medición (en milisegundos).
- echo_us: duración del pulso de eco medido (en microsegundos).
- distancia_cm: distancia calculada en centímetros.
- valido: 1 = medición válida, 0 = medición no válida (sin eco).

Resultado: tabla mediciones_distancia

mediciones_distancia				
id	tiempo_ms	echo_us	distancia_cm	valido
1	0	5824	99.8	1
2	200	3440	58.9	1
3	400	2480	42.6	1
4	600	1712	29.4	1
5	800	1208	20.7	1
6	1000	680	11.7	1
7	1200	0	NULL	0
8	1400	720	12.4	1
9	1600	980	16.9	1
10	1800	1208	20.7	1



Tipos de datos utilizados

- **INTEGER** → números enteros (sin decimales)
- **REAL** → números reales (con decimales)
- **PRIMARY KEY** → asegura valores únicos en la columna id

Introducción a SQL

SQL en sistemas embebidos: consulta y resultados

CONSULTA SQL

```
1 SELECT tiempo_ms, distancia_cm
2 FROM mediciones_distancia
3 WHERE valido = 1
4     AND distancia_cm > 20.0
5 ORDER BY tiempo_ms;
6
```

SELECT : columnas a mostrar

FROM : tabla de origen

WHERE : condición de filtrado

ORDER BY : orden temporal

Tabla: mediciones_distancia

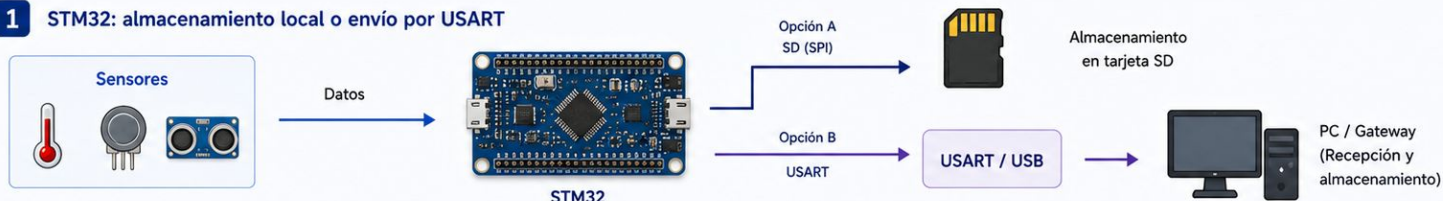
id	tiempo_ms	echo_us	distancia_cm	valido
1	0	1150	19.8	1
2	100	1320	22.7	1
3	200	980	16.8	1
4	300	1450	24.9	1
5	400	2100	36.1	1
6	500	890	15.3	1
7	600	1620	27.8	1
8	700	0	0.0	0
9	800	1750	30.0	1
10	900	1120	19.2	1
11	1000	1980	34.0	1

Filas resaltadas: cumplen valido = 1 y distancia_cm > 20.0

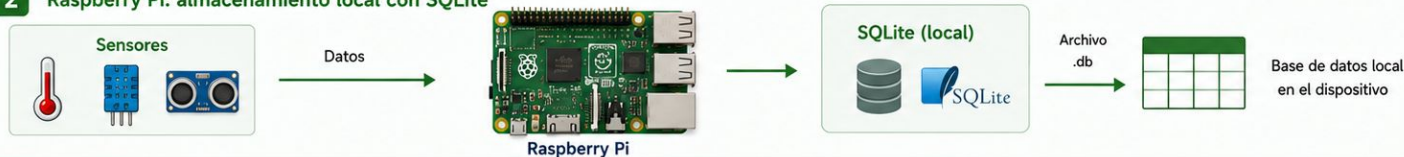
¿Dónde vive la base de datos?

i Los datos generados por sensores pueden almacenarse localmente o enviarse a otros dispositivos o servicios.

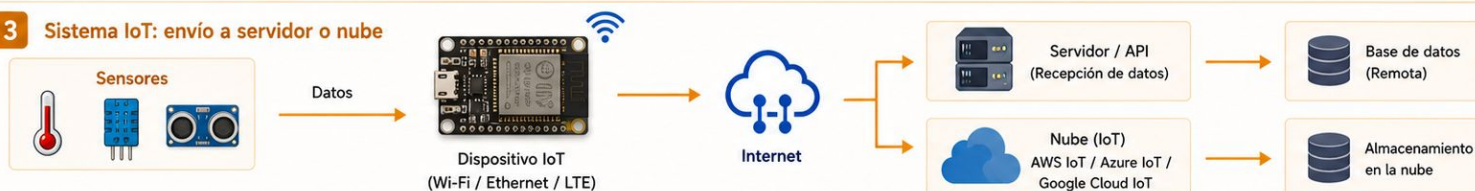
1 STM32: almacenamiento local o envío por USART



2 Raspberry Pi: almacenamiento local con SQLite



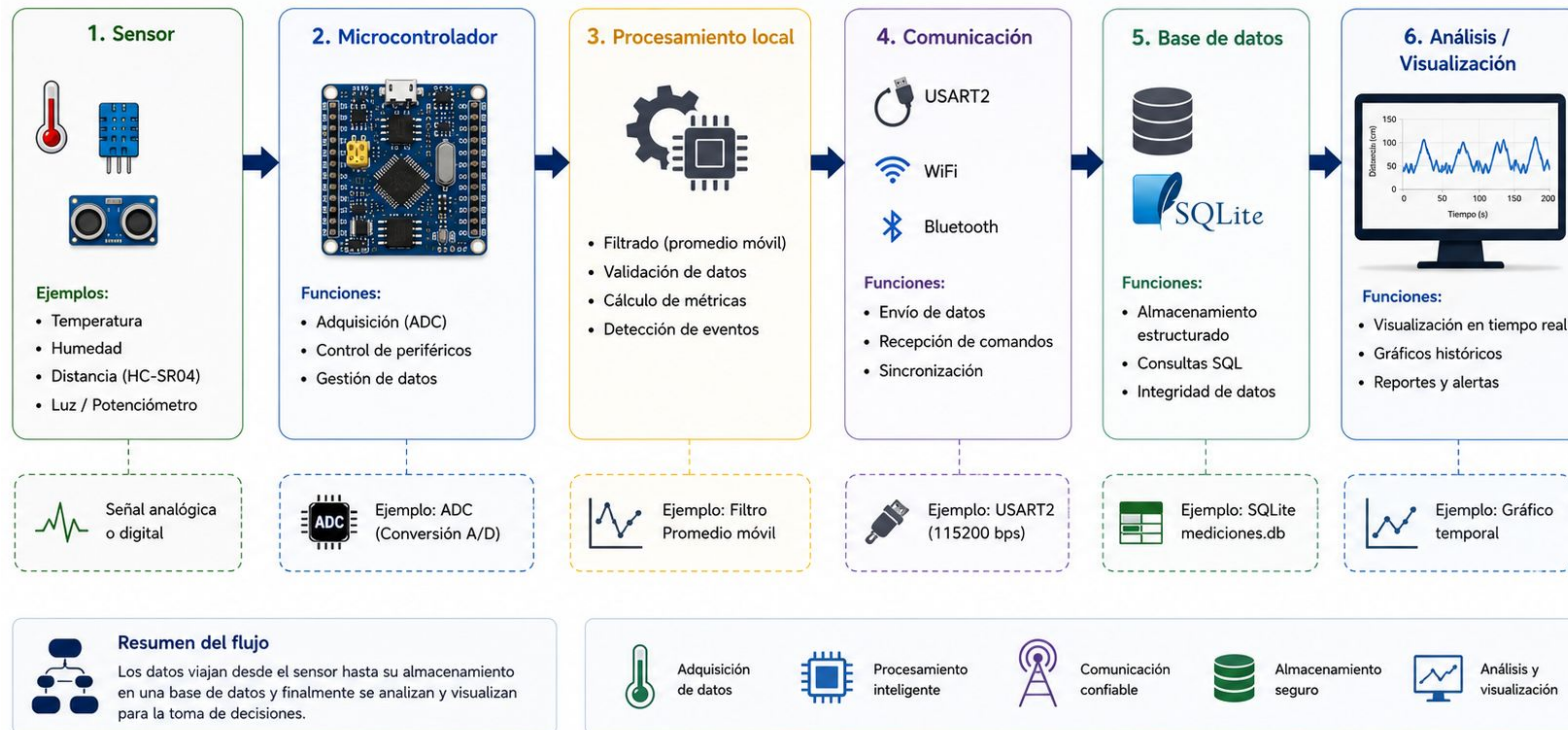
3 Sistema IoT: envío a servidor o nube



i Resumen: Los datos pueden almacenarse en medios locales (SD, SQLite) o en sistemas remotos (servidores o nube) según los requisitos del proyecto.

Arquitectura típica

Flujo completo desde la adquisición de datos hasta su análisis y visualización



Actividad

Diseñar una tabla de base de datos para un sistema embebido



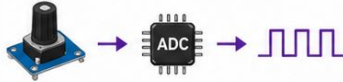
Tu equipo debe definir los campos que se almacenarán, su tipo de dato y una breve descripción.

HC-SR04



Mide distancia
(tiempo de eco)

ADC + PWM



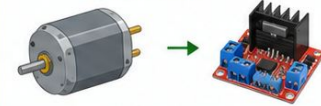
Lee un potenciómetro (ADC)
y genera PWM para control

campo	tipo	descripción



Nombre sugerido para la tabla: _____

Motor DC



Control de velocidad y dirección
mediante PWM

FSM botón + LED



Estados de la máquina finita
y control de LED



1. ¿Qué datos conviene guardar?

- Piensa en las variables y eventos importantes del sistema.
- Considera: tiempo, valores medidos, comandos, estados, resultados.
- ¿Qué información te permitirá analizar el comportamiento del sistema?



2. ¿Qué consultas serían útiles?

- ¿Cómo consultarías los últimos datos?
- ¿Cómo detectarías eventos o condiciones específicas?
- ¿Qué reportes o gráficos te gustaría obtener?



Consejo: Define bien los tipos de dato y las unidades (ej.: cm, ms, %). Esto facilitará las consultas y el análisis.

